

M10 / STUDY COMPANION

M10 Quick Revision Book

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	4
Use after learning, not instead of learning	4
DE10-01 Why containers exist	4
Core recall	4
DE10-02 Images and containers	4
Core recall	4
DE10-03 Writing a Dockerfile	5
Core recall	5
DE10-04 Ports	5
Core recall	5
DE10-05 Volumes and persistence	5
Core recall	5
DE10-06 Networks and service communication	6
Core recall	6
DE10-07 Compose, configuration and secrets	6
Core recall	6
DE10-08 Why orchestration exists	7
Core recall	7
DE10-09 Airflow architecture and UI	7
Core recall	7
DE10-10 DAGs, tasks and dependencies	8

Core recall	8
DE10-11 TaskFlow, operators and task communication	8
Core recall	8
DE10-12 Scheduling and data intervals	8
Core recall	8
DE10-13 Retries, timeouts and failure behavior	9
Core recall	9
DE10-14 Parameters and task communication	9
Core recall	9
DE10-15 Connections, variables and secrets	10
Core recall	10
DE10-16 Sensors and waiting	10
Core recall	10
DE10-17 Logs and debugging	11
Core recall	11
DE10-18 Catchup, backfills and reprocessing	11
Core recall	11
DE10-19 Integration and production-style orchestration	11
Core recall	11

Read this first

Use after learning, not instead of learning

Recall each lesson from the compact card, then close this book and reproduce the command/design from memory. Any weak card routes back to only that learner-book lesson.

DE10-01 | Why containers exist

Core recall

Containers improve reproducibility only when the build is reproducible, the configuration boundary is explicit, and persistent state is handled separately. A containerized pipeline can still be non-idempotent, insecure or logically wrong.

Failure to remember

A common mistake is saying 'the container includes its own operating system.' Linux containers usually share the host kernel. Another mistake is treating a container as durable storage. If business data exists only in a disposable writable layer, removing the container can remove that state.

One retrieval prompt

A colleague says, 'We use Docker, so dependency pinning no longer matters.' Refute the statement using the difference between an image recipe and an unpinned package install inside that recipe.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-02 | Images and containers

Core recall

Promotion usually moves immutable image candidates between environments while environment-specific configuration changes outside the image. This makes rollback and audit much easier than hand-editing running machines.

Failure to remember

Do not debug every container problem by rebuilding. A wrong environment variable is a runtime-config problem. Conversely, editing a host source file will not change an already-built image unless that file is bind-mounted or the image is rebuilt.

One retrieval prompt

Imagine two containers created from the same image produce different output because they were given different `SOURCE_URL` values. Explain why this does not violate image immutability.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-03 | Writing a Dockerfile

Core recall

A professional Dockerfile makes the runtime boundary reviewable: known base, minimal context, deterministic dependency install, explicit work directory, non-root user and a clear foreground process.

Failure to remember

Do not COPY . . blindly from a large project that contains .env, virtual environments or evidence files. Do not put passwords in ENV or RUN instructions; image history and layers can preserve them. Do not use root merely because it avoids permissions learning.

One retrieval prompt

Add a requirements file containing one package. Reorder the Dockerfile so dependency metadata is copied/installed before frequently changing application source. Explain the cache benefit without claiming the cache proves packages are pinned.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-04 | Ports

Core recall

Expose only interfaces that need host/external access. Databases and internal services often communicate on private container networks without publishing every port to the host.

Failure to remember

EXPOSE 8000 does not publish port 8000. Publishing 8090:8000 also does not make the source process listen on 8090 inside the container. If a process listens only on 127.0.0.1 inside some container scenarios, publication may still not behave as you expect; always inspect the actual bind/listen configuration.

One retrieval prompt

Port 8090 is already occupied on your laptop. Design the smallest safe correction and explain why rebuilding the image would be unrelated to the conflict.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-05 | Volumes and persistence

Core recall

State ownership is an architectural decision. Volumes survive container replacement but do not by themselves provide backup, replication, retention or corruption recovery.

Failure to remember

Never teach `docker compose down -v` as routine cleanup when the volume contains data you care about. Also avoid bind-mounting broad host folders with write access just to bypass permissions. Mount only what the service needs.

One retrieval prompt

A teammate says 'the database is in a container, so backups are unnecessary because the volume persists.' Explain why persistence and backup/recovery are different controls.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-06 | Networks and service communication

Core recall

Stable service discovery plus health/readiness checks is safer than coding around ephemeral addresses. Network boundaries also help you avoid publishing internal-only services to the host.

Failure to remember

Do not fix a service-to-service failure by copying the current container IP into source code. Do not assume `depends_on` means the dependency is ready to serve; start order and readiness are different concepts. A health check or application retry may still be necessary.

One retrieval prompt

Rename the Compose service from source to feed. List every configuration reference that should change and every place that should not need an IP update.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-07 | Compose, configuration and secrets

Core recall

Version-control safe structure and names; inject environment-owned values at runtime; store real secrets in a secrets mechanism appropriate to the deployment environment. Reproducibility means a clean environment can rebuild the same topology from documented inputs.

Failure to remember

Never commit real passwords merely because they are in `.env`. `.env` is configuration storage, not automatically a secret vault. Do not use Compose Variables as a substitute for production secret backends. Also do not call `down -v` casually.

One retrieval prompt

Move the host port and output volume name through environment interpolation. Explain why changing those values should not require rebuilding the worker image.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-08 | Why orchestration exists

Core recall

Orchestration adds controlled execution semantics around testable units. A good DAG is a readable dependency model, not a dumping ground for every line of business code.

Failure to remember

Do not move all transformation code into a huge DAG file just because Airflow can run Python. Do not make orchestration responsible for fixing bad source semantics. Keep the business work testable outside the scheduler where possible.

One retrieval prompt

The pipeline now has two independent downstream reports after validation. Redraw the graph and explain why the orchestrator can run those two branches concurrently without changing the validation contract.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-09 | Airflow architecture and UI

Core recall

Operational diagnosis improves when you name the layer first: definition/parse, scheduling, task execution, dependency/service, or data correctness. Airflow gives state; your engineering still has to interpret it.

Failure to remember

When a Dag is missing, inspect import/parsing errors before reinstalling everything. When a task fails, distinguish Dag parse failure from task runtime failure. UI red does not tell you root cause; logs and task context do.

One retrieval prompt

The scheduler is healthy but a provider import inside one Dag is missing. Explain why restarting the whole machine is a weaker first response than checking Dag import errors and the environment where Airflow parses files.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-10 | DAGs, tasks and dependencies

Core recall

Graph design is operational design: task boundaries determine what can retry independently, what can run concurrently, what evidence is isolated and how failures block downstream side effects.

Failure to remember

Do not create fake dependencies to force a pretty left-to-right diagram. Do not split work into tiny tasks that pass huge data through orchestration metadata. Task boundaries should reflect retries, ownership, resources, observability or meaningful dependency boundaries.

One retrieval prompt

A new alert task should run only after validation failure, not as an ordinary downstream success dependency. Describe the trigger/branching requirement rather than connecting it blindly to publish.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-11 | TaskFlow, operators and task communication

Core recall

Use orchestration metadata for coordination; use data stores for data. This keeps scheduler/metadata systems small, makes retries clearer and avoids coupling workflow correctness to oversized XCom serialization.

Failure to remember

Do not confuse Python function call appearance with ordinary eager execution at Dag-definition time. Do not return huge datasets just because it works in a toy local environment. Do not use a global Variable to pass per-run task output.

One retrieval prompt

Replace a returned full list of 100,000 IDs with an object-storage/database path plus count/checksum metadata. Explain how this improves operational robustness.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-12 | Scheduling and data intervals

Core recall

Interval-correct workflows make backfills, retries and audits possible. Time semantics are part of data correctness, not a scheduler trivia question.

Failure to remember

`date.today()` is wrong for replayable partition selection. An offset-naive start date creates ambiguity. Changing catchup without understanding what historical runs will be created can unexpectedly process or skip partitions.

One retrieval prompt

Change from daily to hourly processing. State what one data interval represents and how the partition path needs more than a date to stay unique.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-13 | Retries, timeouts and failure behavior

Core recall

Retry policy is part of side-effect design. Reliable orchestration distinguishes transient infrastructure failures from deterministic data/config defects and leaves enough evidence to diagnose both.

Failure to remember

Do not catch every exception and return success. Do not retry authentication errors or schema violations indefinitely. Do not add retries around a non-idempotent write without first defining duplicate/transaction behavior.

One retrieval prompt

A publish task sends an external payment request and you cannot prove idempotency. Decide whether automatic retry is safe and name the external idempotency key/transaction control you would require.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-14 | Parameters and task communication

Core recall

Parameters make controlled variation explicit. Keep them small, typed and business-readable; keep observed run facts in task outputs/metadata and bulk data in data stores.

Failure to remember

Do not read manually supplied Param values from `dag.params` inside top-level definition code and assume you have run-time overrides; Airflow documentation distinguishes defaults from `DagRun/task` context. Do not store per-run task outputs in global Variables.

One retrieval prompt

A teammate wants a Param containing 50,000 customer IDs. Redesign the interface so the Param contains a durable input reference plus validation metadata instead.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-15 | Connections, variables and secrets

Core recall

Portable orchestration keeps code stable while environment-owned connection/secrets configuration changes behind stable identifiers. Least privilege and log sanitization remain required.

Failure to remember

If a secret was committed, moving it to a Variable after the fact does not erase Git history; rotate it and follow repository history-removal policy. Do not print connection URIs/tokens into task logs. Do not confuse environment interpolation with secret protection.

One retrieval prompt

Your company moves from Airflow DB-stored Connections to Vault. Explain why well-designed DAG code using the same connection ID may not need business-logic changes.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-16 | Sensors and waiting

Core recall

Waiting is capacity planning. A correct sensor defines condition, cadence, timeout, resource mode and failure ownership; otherwise a quiet wait can be as operationally expensive as a failed task.

Failure to remember

Do not write `while True: sleep(...)` inside an ordinary worker task as your default waiting strategy. Do not use a 1-second poke interval for a file expected once per hour. Do not omit timeout and create endless invisible waiting.

One retrieval prompt

The external platform can emit an event instead of being polled. Explain why event-driven/deferrable waiting may be better than frequent polling and what failure/timeout behavior still needs definition.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-17 | Logs and debugging

Core recall

Operationally useful logging connects workflow state to data evidence. Preserve failed attempts, sanitize sensitive values, and design logs so another engineer can diagnose without reproducing your entire local setup.

Failure to remember

Do not log entire dataframes or response bodies containing PII/secrets merely because debugging is hard. Do not erase failed logs/evidence after repair. Do not read only the final exception line when the earlier stack/context shows the actual cause.

One retrieval prompt

A task is green but downstream total is wrong. Explain why the debugging path should move from orchestration status to data reconciliation/logic evidence rather than adding more retries.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-18 | Catchup, backfills and reprocessing

Core recall

Historical recovery is a first-class reliability feature. Partition identity, side-effect idempotency and explicit range controls determine whether backfill restores trust or creates duplicate damage.

Failure to remember

Do not backfill a non-idempotent external write without duplicate/transaction controls. Do not let historical runs overwrite a shared `latest.csv`. Do not assume `catchup=True` is always desirable; choose based on whether missed historical intervals should run.

One retrieval prompt

A downstream API cannot safely accept the same event twice. Introduce an idempotency key or staging/upsert boundary and explain how that changes backfill safety.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.

DE10-19 | Integration and production-style orchestration

Core recall

M10 mastery means you can defend both layers: environment reproducibility and workflow reliability. The G04 gate tests fresh pipeline reasoning after this module; it is not passed by rereading the final lesson.

Failure to remember

Do not use a passing static parser as evidence that Docker networking works. Do not use a green Airflow task as evidence that totals are correct. Do not hide environment blockers; record them and continue with the parts that can be truthfully validated.

One retrieval prompt

The business switches from daily CSV to hourly object-store files. Identify which Docker pieces can remain unchanged, which Dag schedule/partition logic changes, and which validation/backfill controls must be retested.

Refresh visual/reference [Open mapped source](#)

Open only if recall remains weak; then close it and retry from memory.